

Exercice 02 — TEST-5 : Suite E2E structuree en Page Object Model pour la plateforme

Projet fil rouge : Code un max - Suite de tests automatisés de la plateforme de cours

Cet exercice **s'ajoute au travail précédent**. Vous continuez à construire le projet étape par étape.

Dans cet exercice : Au chapitre 1, l'equipe QA de Code un max a mis en place l'environnement (pytest, selenium, un venv) et ecrit quelques tests E2E de survie sur la plateforme de cours. Ces tests fonctionnent mais chaque scenario recrit le driver, redeclare les memes locators et duplique le login. Impossible a maintenir des que l'equipe front change un bouton.

Avant de commencer

L'**Exemple 02** ([exemples/exemple-02/exemple-02.md](#)) t'a fait pratiquer la technique sur un cas neutre. Ici, tu l'appliques au projet **Code un max - Suite de tests automatisés de la plateforme de cours**. Assure-toi de l'avoir lu et suivi avant de commencer.

Vous devez aussi avoir **terminé l'Exercice 01**, car cet exercice s'appuie sur le travail déjà réalisé.

Objectif

Transformer les premiers tests E2E du chapitre 1 en une suite propre de 8 a 10 tests structures en Page Object Model, relancable en local, couvrant les parcours cles de la plateforme de cours Code un max.

Prérequis

- Python 3.11+ installé
- Git
- VS Code
- Chrome ou Firefox
- Compte GitHub
- Bases de la programmation Python
- Notions de développement web
- Avoir complété les exercices précédents du projet fil rouge

Scénario

Le lead QA de Code un max ouvre le ticket TEST-5 : 'Notre poignee de tests E2E devient ingerable. Il faut une base solide : navigateur en fixture, login mutualise, locators centralises en Page Objects, et une couverture des 5 parcours cles (login valide, login refuse, navigation catalogue, inscription a un cours via formulaire, deconnexion). Objectif : 8 a 10 tests qui passent en local d'un seul pytest.' La plateforme de test tourne en local (par exemple <http://localhost:8000>, pages /login, /catalogue, /cours//inscription, /dashboard).

Étapes à réaliser

Étape 1 — Poser l'arborescence du projet de tests

Créer une structure claire qui sépare les Page Objects, les tests et la configuration partagée.

Ce que vous devez faire :

- Créer un dossier pages/ avec un **init.py**, et un dossier tests/ pour les scénarios.
- Prévoir un conftest.py à la racine pour les fixtures partagées.
- Ajouter un pytest.ini ou pyproject définissant testpaths et une convention de nommage.

Résultat attendu : Une arborescence où pytest collecte les tests de tests/ sans erreur d'import de pages/.

Piste : Un **init.py** vide dans pages/ suffit à en faire un package importable depuis les tests.

Étape 2 — Centraliser le navigateur dans une fixture

Déplacer la création et la fermeture du driver dans conftest.py pour que plus aucun test ne l'instancie.

Ce que vous devez faire :

- Écrire une fixture 'driver' qui construit le navigateur en mode headless et le détruit après le test.
- Utiliser yield pour garantir la fermeture même en cas d'échec.
- Choisir une seule stratégie d'attente pour éviter les timeouts qui s'additionnent.

Résultat attendu : Aucun webdriver.Chrome() ne subsiste dans les fichiers de test ; ils reçoivent 'driver' en argument.

Erreur fréquente : Placer driver.quit() à la fin du test au lieu de la fixture : si le test échoue avant, le navigateur reste ouvert et les runs suivants s'accumulent.

Étape 3 — Écrire la BasePage

Regrouper les interactions Selenium bas niveau dans une classe réutilisable par toutes les pages.

Ce que vous devez faire :

- Définir une classe BasePage qui reçoit le driver dans son constructeur.
- Y placer des méthodes génériques : ouvrir une URL, saisir du texte, cliquer, lire un texte, vérifier la présence.
- Utiliser des attentes explicites plutôt que des time.sleep.

Résultat attendu : Une classe qui manipule le driver sans jamais connaître un locator spécifique à une page.

Piste : Les méthodes prennent le locator en paramètre sous forme de tuple (By.X, 'valeur'), pas en dur.

Étape 4 — Modéliser les pages de la plateforme

Créer un Page Object par écran clé : LoginPage, CataloguePage, InscriptionPage, DashboardPage.

Ce que vous devez faire :

- Dans chaque page, déclarer les locators comme attributs de classe.
- Exposer des methodes métier : login(user, pwd), ouvrir_catalogue(), s_inscrire_au_cours(nom), se_deconnecter().
- Faire heriter chaque page de BasePage.
- Faire renvoyer par les actions de navigation le Page Object de la page de destination quand c'est pertinent.

Résultat attendu : Quatre classes de pages où seuls les locators changent d'une page à l'autre, la mécanique venant de BasePage.

Référence : Voir le tutoriel du chapitre pour le pattern LoginPage(BasePage) et le chaînage .open().login(...).

Étape 5 — Mutualiser le login via une fixture dédiée

Beaucoup de scénarios exigent d'être déjà connecté. Fournir une fixture qui livre un navigateur déjà authentifié.

Ce que vous devez faire :

- Créer une fixture 'session_connectee' qui s'appuie sur la fixture driver et effectue le login avec un compte de test valide.
- La fixture renvoie soit le driver connecté, soit directement le DashboardPage.
- Les tests qui partent du tableau de bord consomment cette fixture au lieu de refaire le login.

Résultat attendu : Les tests de navigation, inscription et déconnexion ne contiennent plus le code de login.

Erreur fréquente : Coder les identifiants en dur dans dix tests : centralise-les dans la fixture (ou une variable de config) pour n'avoir qu'un endroit à changer.

Étape 6 — Écrire les 5 scénarios E2E

Couvrir les parcours clés en visant 8 à 10 tests au total grâce au paramétrage.

Ce que vous devez faire :

- Login valide : vérifier l'arrivée sur le tableau de bord.
- Login refuse : paramétrer plusieurs jeux invalides (mauvais mot de passe, utilisateur inconnu, champs vides) et vérifier le message adéquat.
- Navigation catalogue : vérifier qu'au moins un cours attendu est présent.
- Inscription via formulaire : remplir et soumettre, vérifier le message de confirmation.
- Déconnexion : partir de session_connectee, se déconnecter, vérifier le retour à l'écran de login.
- Structurer chaque test en Arrange / Act / Assert avec des messages d'assert explicites.

Résultat attendu : Entre 8 et 10 tests collectés (le login refuse paramètre en genre plusieurs), tous verts en local.

Piste : Le paramétrage du login refuse fait grimper le compte de tests sans dupliquer de code.

Étape 7 — Vérifier l'indépendance et le nommage

S'assurer que la suite est fiable quel que soit l'ordre et lisible pour un nouvel arrivant.

Ce que vous devez faire :

- Lancer la suite avec l'option qui melange l'ordre des tests et verifier qu'ils passent toujours.
- Verifier qu'aucun test ne depend d'un etat laisse par un autre.
- Relire les noms de tests : ils doivent decrire le comportement attendu, pas l'implementation.

Résultat attendu : La suite passe en ordre aleatoire ; les noms de tests se lisent comme des specifications.

Référence : installer `pytest-randomly` (`pip install pytest-randomly`) puis relancer `pytest` — le melange est actif par default et la seed s'affiche en entete. Utiliser `-p no:randomly` uniquement pour retrouver l'ordre fixe lors du debogage.

Vérification

Quand vous avez terminé, vérifiez que :

- ☐ Un seul 'pytest' a la racine collecte et execute 8 a 10 tests, tous verts.
- ☐ Aucun fichier de test ne contient `webdriver.Chrome()` ni un locator en dur.
- ☐ Changer un locator ne demande de modifier qu'un seul Page Object.
- ☐ La suite reste verte quand on change l'ordre d'execution.
- ☐ Chaque assert affiche une valeur utile en cas d'echec.

En pratique

La fixture de login connectee est ce qui fait vraiment gagner du temps sur ce genre de suite, mais attention : si le login lui-meme casse, tous les tests qui en dependent tombent d'un coup et le rapport devient illisible. Garde toujours un `test_login_valide` isole qui n'utilise pas cette fixture, pour distinguer 'le login est casse' de 'la deconnexion est cassee'.

Bonus (Facultatif mais Recommandé)

Ajouter une introduction `pytest-bdd` : reecrire le scenario de login valide en Gherkin (Given/When/Then) et le brancher sur les memes Page Objects, pour montrer que le POM se reutilise quel que soit le style de test.

Livrable attendu

Un depot avec `pages/` (`base_page`, `login_page`, `catalogue_page`, `inscription_page`, `dashboard_page`), `tests/` (5 scenarios), `conftest.py` (fixtures `driver` et `session_connectee`), un fichier de config `pytest`, et un court README expliquant comment lancer la suite. Le tout attache au ticket TEST-5.

Une fois terminé, consultez la **correction** dans [corrections-exercices/correction-exercice-02/correction-exercice-02.md](#) pour comparer votre travail.